

Migrations

El Museo de Historia Natural está investigando los patrones de migración de los dinosaurios en Bolivia. Los Paleontólogos han descubierto huellas de dinosaurios en N sitios diferentes, enumerados del 0 al $N - 1$ en orden antigüedad decreciente: el sitio 0 contiene a las huellas más antiguas, mientras que el sitio $N - 1$ contiene a las más jóvenes.

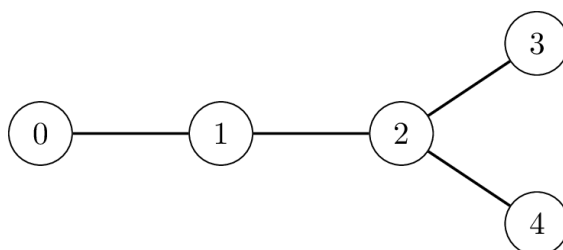
Los dinosaurios migraron a cada sitio (excepto al sitio 0) de un sitio específico y más antiguo. Para cada sitio i con $1 \leq i \leq N - 1$, existe un único sitio más antiguo $P[i]$ (con $P[i] < i$) tal que algunos dinosaurios migraron directamente desde el sitio $P[i]$ hasta el sitio i . Un sitio antiguo pudo haber sido fuente de migraciones hacia varios sitios más jóvenes.

Los paleontólogos modelan cada migración como una **conexión no dirigida** entre los sitios i y $P[i]$. Note que para cada par de sitios diferentes x e y , es posible viajar desde x hacia y atravesando una secuencia de conexiones.

La **distancia** entre los sitios x e y es definida como el mínimo número de conexiones requeridas para viajar desde x hacia y .

Por ejemplo, la siguiente imagen muestra un caso con $N = 5$ sitios, donde $P[1] = 0$, $P[2] = 1$, $P[3] = 2$, y $P[4] = 2$.

Uno puede viajar del sitio 3 al sitio 4 usando 2 conexiones, así que la distancia entre ellos es 2.



El Museo apunta a identificar un par de sitios con la máxima distancia posible. Note que la respuesta no necesariamente es única: por ejemplo, en el ejemplo anterior, tanto los pares $(0, 3)$ como $(0, 4)$ tienen una distancia de 3, que es la máxima. En ambos casos, cualquiera de los pares válidos es considerado como respuesta correcta.

Inicialmente los valores de $P[i]$ son **desconocidos**. El Museo envía un equipo de investigación para visitar los sitios $1, 2, \dots, N - 1$, en orden. Al llegar a cada sitio i ($1 \leq i \leq N - 1$), el equipo realiza

las siguientes 2 acciones:

- Determina el valor de $P[i]$, esto es, la fuente de migración del sitio i .
- Decide **si enviar o no** un mensaje al Museo

Los mensajes se transmiten a través de un costoso sistema de comunicación por satélite, por lo que cada mensaje debe ser un entero positivo entre 1 y 20 000, inclusive. Además, el equipo tiene permitido enviar **como máximo 50 mensajes** en total.

Tu tarea es implementar una estrategia de modo que:

- El equipo de investigación seleccione los sitios desde los que se envían los mensajes, junto con el valor de cada mensaje.
- El Museo pueda determinar un par de sitios con la máxima distancia, únicamente basado en los mensajes que recibió del equipo de investigación.

Enviar grandes números por medio del satélite es costoso. Tu puntaje dependerá tanto del mayor número enviado, y el número total de mensajes transmitidos.

Detalles de la Implementación

Deberás implementar dos funciones; una para el equipo de investigación, y otra para el Museo.

La función que deberías implementar para el **equipo de investigación** es:

```
int send_message(int N, int i, int Pi)
```

- N : cantidad de sitios con huellas
- i : El índice del sitio actual que el equipo está visitando
- Pi : el valor de $P[i]$.
- Esta función es llamada $N - 1$ veces para cada test case, siguiendo el orden $i = 1, 2, \dots, N - 1$.

Esta función deberá devolver $S[i]$ especificando la acción tomada por el equipo de investigación en el sitio i :

- $S[i] = 0$: el equipo de investigación decide no enviar un mensaje desde el sitio i .
- $1 \leq S[i] \leq 20\,000$: el equipo de investigación envía el número $S[i]$ como mensaje desde el sitio i .

La función que deberías implementar para el **Museo** es:

```
std::pair<int,int> longest_path(std::vector<int> S)
```

- S : un arreglo de longitud N tal que:

- $S[0] = 0$.
- Para cada $1 \leq i \leq N - 1$, $S[i]$ es el entero devuelto por `send_message(N, i, Pi)`.
- Esta función es llamada exactamente una vez por cada test case.

Esta función deberá devolver un par de sitios (U, V) con la máxima distancia.

En el grading oficial, el programa que llama a las funciones anteriores se ejecuta **exactamente dos veces**.

- Durante la primera ejecución del programa:
 - `send_message` es llamada exactamente $N - 1$ veces.
 - **Tu programa puede almacenar y retener información en llamadas sucesivas**
 - Los valores devueltos son almacenados por el grading system.
 - En algunos casos, el comportamiento del grader es **adaptativo**. Esto significa que el valor de $P[i]$ en una llamada a `send_message` puede depender de las acciones tomadas por el equipo de investigación durante las llamadas anteriores
- Durante la segunda ejecución:
 - `longest_path` es llamada exactamente una vez, con los valores almacenados por el grading system durante la primera ejecución.

Restricciones

- $N = 10\,000$
- $0 \leq P[i] < i$ para cada i tal que $1 \leq i \leq N - 1$.

Subtareas y Puntuación

Subtarea	Puntaje	Restricciones Adicionales
1	30	El sitio 0 y algún otro sitio tienen la máxima distancia entre todos los pares de sitios
2	70	Sin restricción adicional

Sea Z el máximo entero que aparece en el arreglo S , y sea M la cantidad de mensajes enviados por el equipo de investigación

En cualquiera de los test cases, si al menos una de las siguientes condiciones se cumple, entonces el puntaje de tu solución para ese test case será 0 (reportado como `Output isn't correct` en CMS)

- Al menos un elemento de S es inválido
- $Z > 20\,000$ o $M > 50$.
- EL valor de retorno de la llamada a la función `longest_path` es incorrecta

Caso contrario, el puntaje de la subtarea 1 es calculada de la siguiente manera:

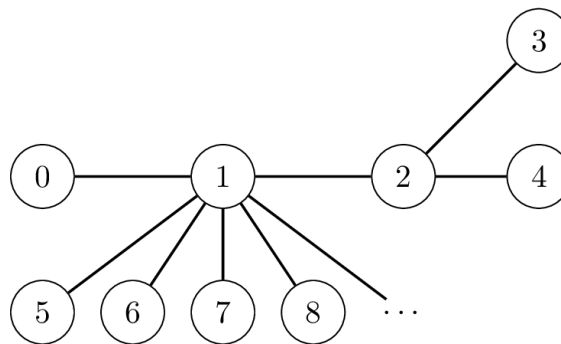
Condición	Puntaje
$9\,998 \leq Z \leq 20\,000$	10
$102 \leq Z \leq 9997$	16
$5 \leq Z \leq 101$	23
$Z \leq 4$	30

El puntaje de la subtarea 2 es calculada de la siguiente manera:

Condición	Puntaje
$5 \leq Z \leq 20\,000$	$35 - 25 \log_{4000} \left(\frac{Z}{5} \right)$
$Z \leq 4$ and $32 \leq M \leq 50$	40
$Z \leq 4$ and $8 < M < 32$	$70 - 30 \log_4 \left(\frac{M}{8} \right)$
$Z \leq 4$ and $M \leq 8$	70

Ejemplo

Sea $N = 10\,000$. Considera la situación donde $P[1] = 0$, $P[2] = 1$, $P[3] = 2$, $P[4] = 2$, y $P[i] = 1$ para $i > 4$.



Suponga que la estrategia del equipo de investigación es enviar el mensaje $10 \cdot V + U$ cada que el par de sitios (U, V) con máxima distancia cambie como resultado de una llamada `send_message`.

Inicialmente, el par con máxima distancia es $(U, V) = (0, 0)$. Considere la siguiente secuencia de llamadas durante la primera ejecución del programa:

Llamada a función	(U, V)	Valor de retorno
<code>send_message(10000, 1, 0)</code>	$(0, 1)$	10
<code>send_message(10000, 2, 1)</code>	$(0, 2)$	20
<code>send_message(10000, 3, 2)</code>	$(0, 3)$	30
<code>send_message(10000, 4, 2)</code>	$(0, 3)$	0

Note que en todas las llamadas restantes el valor de $P[i]$ es 1. Esto significa que el par con la máxima distancia no cambiará, así que el equipo no enviará más mensajes.

Luego, en la segunda ejecución del programa, se realizará la siguiente llamada:

```
longest_path([0, 10, 20, 30, 0, ...])
```

El museo lee el último mensaje, que es $S[3] = 30$, y deduce que $(0, 3)$ es un par de sitios con la máxima distancia. Así que la llamada devolverá $(0, 3)$.

Note que esta estrategia no siempre permitirá al Museo determinar correctamente un par con la máxima distancia.

Grader de Ejemplo

El grader de ejemplo llama a `send_message` y `longest_path` en la misma ejecución, lo cual es diferente del grader oficial.

Formato de Input:

```
N
P[1] P[2] ... P[N-1]
```

Formato de Output:

```
S[1] S[2] ... S[N-1]
U V
```

Note que puedes usar el grader de ejemplo con cualquier valor de N